C++ Math Functions (<cmath>)

By Dimpal Baishya

Abstract

The <cmath> library provides a comprehensive collection of functions for performing mathematical operations. Below is a structured, production-ready reference guide complete with working examples for every function.

Contents

1 Absolute Values

1.1 abs(x)

Description: Returns the absolute value of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    int x = -10;
    auto result = std::abs(x);
    std::cout << "abs(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::abs`, and outputs the result. Use this when you need to mathematically evaluate the absolute value of x.

1.2 fabs(x)

Description: Returns the absolute value of a floating x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = -10.5;
    auto result = std::fabs(x);
    std::cout << "fabs(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::fabs`, and outputs the result. Use this when you need to mathematically evaluate the absolute value of a floating x.

2 Trigonometric Functions

2.1 acos(x)

Description: Returns the arccosine of x, in radians

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 0.5;
    auto result = std::acos(x);
    std::cout << "acos(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::acos`, and outputs the result. Use this when you need to mathematically evaluate the arccosine of x, in radians.

2.2 asin(x)

Description: Returns the arcsine of x, in radians

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 0.5;
    auto result = std::asin(x);
    std::cout << "asin(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::asin`, and outputs the result. Use this when you need to mathematically evaluate the arcsine of x, in radians.

2.3 atan(x)

Description: Returns the arctangent of x as a numeric value between $-\pi/2$ and $\pi/2$ radians

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::atan(x);
    std::cout << "atan(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::atan`, and outputs the result. Use this when you need to mathematically evaluate the arctangent of x as a numeric value between $-\pi/2$ and $\pi/2$ radians.

2.4 atan2(y, x)

Description: Returns the angle theta from the conversion of rectangular coordinates (x, y) to polar coordinates (r, θ)

```cpp
#include <iostream>
#include <cmath>

int main() {
    double y = 10.0, x = 10.0;
    auto result = std::atan2(y, x);
    std::cout << "atan2(y, x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::atan2`, and outputs the result. Use this when you need to mathematically evaluate the angle theta from the conversion of rectangular coordinates (x, y) to polar coordinates (r, θ).

2.5 cos(x)

Description: Returns the cosine of x (x is in radians)

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 3.14159 / 3.0;
    auto result = std::cos(x);
    std::cout << "cos(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::cos`, and outputs the result. Use this when you need to mathematically evaluate the cosine of x (x is in radians).

2.6 sin(x)

Description: Returns the sine of x (x is in radians)

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 3.14159 / 6.0;
    auto result = std::sin(x);
    std::cout << "sin(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::sin`, and outputs the result. Use this when you need to mathematically evaluate the sine of x (x is in radians).

2.7 tan(x)

Description: Returns the tangent of x (x is in radians)

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 3.14159 / 4.0;
    auto result = std::tan(x);
    std::cout << "tan(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::tan`, and outputs the result. Use this when you need to mathematically evaluate the tangent of x (x is in radians).

3 Hyperbolic Functions

3.1 acosh(x)

Description: Returns the hyperbolic arccosine of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0;
    auto result = std::acosh(x);
    std::cout << "acosh(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::acosh`, and outputs the result. Use this when you need to mathematically evaluate the hyperbolic arccosine of x.

3.2 asinh(x)

Description: Returns the hyperbolic arcsine of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0;
    auto result = std::asinh(x);
    std::cout << "asinh(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::asinh`, and outputs the result. Use this when you need to mathematically evaluate the hyperbolic arcsine of x.

3.3 atanh(x)

Description: Returns the hyperbolic arctangent of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 0.5;
    auto result = std::atanh(x);
    std::cout << "atanh(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::atanh`, and outputs the result. Use this when you need to mathematically evaluate the hyperbolic arctangent of x.

3.4 cosh(x)

Description: Returns the hyperbolic cosine of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::cosh(x);
    std::cout << "cosh(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::cosh`, and outputs the result. Use this when you need to mathematically evaluate the hyperbolic cosine of x.

3.5 sinh(x)

Description: Returns the hyperbolic sine of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::sinh(x);
    std::cout << "sinh(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::sinh`, and outputs the result. Use this when you need to mathematically evaluate the hyperbolic sine of x.

3.6 tanh(x)

Description: Returns the hyperbolic tangent of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::tanh(x);
    std::cout << "tanh(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::tanh`, and outputs the result. Use this when you need to mathematically evaluate the hyperbolic tangent of x.

4 Exponential and Logarithmic Functions

4.1 exp(x)

Description: Returns the value of e^x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0;
    auto result = std::exp(x);
    std::cout << "exp(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::exp`, and outputs the result. Use this when you need to mathematically evaluate the value of e^x.

4.2 exp2(x)

Description: Returns the value of 2^x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 3.0;
    auto result = std::exp2(x);
    std::cout << "exp2(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::exp2`, and outputs the result. Use this when you need to mathematically evaluate the value of 2^x.

4.3 expm1(x)

Description: Returns $e^x - 1$

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::expm1(x);
    std::cout << "expm1(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::expm1`, and outputs the result. Use this when you need to mathematically evaluate $e^x - 1$.

4.4 log(x)

Description: Returns the natural logarithm of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.71828;
    auto result = std::log(x);
    std::cout << "log(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::log`, and outputs the result. Use this when you need to mathematically evaluate the natural logarithm of x.

4.5 log10(x)

Description: Returns the base 10 logarithm of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 100.0;
    auto result = std::log10(x);
    std::cout << "log10(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::log10`, and outputs the result. Use this when you need to mathematically evaluate the base 10 logarithm of x.

4.6 log1p(x)

Description: Returns the natural logarithm of $x + 1$

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 0.0;
    auto result = std::log1p(x);
    std::cout << "log1p(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::log1p`, and outputs the result. Use this when you need to mathematically evaluate the natural logarithm of $x + 1$.

4.7 log2(x)

Description: Returns the base 2 logarithm of the absolute value of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 8.0;
    auto result = std::log2(x);
    std::cout << "log2(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::log2`, and outputs the result. Use this when you need to mathematically evaluate the base 2 logarithm of the absolute value of x.

4.8 logb(x)

Description: Returns the floating-point base logarithm of the absolute value of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 123.45;
    auto result = std::logb(x);
    std::cout << "logb(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::logb`, and outputs the result. Use this when you need to mathematically evaluate the floating-point base logarithm of the absolute value of x.

4.9 ilogb(x)

Description: Returns the integer part of the floating-point base logarithm of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 123.45;
    auto result = std::ilogb(x);
    std::cout << "ilogb(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::ilogb`, and outputs the result. Use this when you need to mathematically evaluate the integer part of the floating-point base logarithm of x.

5 Power and Root Functions

5.1 cbrt(x)

Description: Returns the cube root of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 27.0;
    auto result = std::cbrt(x);
    std::cout << "cbrt(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::cbrt`, and outputs the result. Use this when you need to mathematically evaluate the cube root of x.

5.2 pow(x, y)

Description: Returns the value of x to the power of y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0, y = 3.0;
    auto result = std::pow(x, y);
    std::cout << "pow(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::pow`, and outputs the result. Use this when you need to mathematically evaluate the value of x to the power of y.

5.3 sqrt(x)

Description: Returns the square root of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 16.0;
    auto result = std::sqrt(x);
    std::cout << "sqrt(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::sqrt`, and outputs the result. Use this when you need to mathematically evaluate the square root of x.

5.4 hypot(x, y)

Description: Returns $\sqrt{x^2 + y^2}$ without intermediate overflow or underflow

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 3.0, y = 4.0;
    auto result = std::hypot(x, y);
    std::cout << "hypot(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::hypot`, and outputs the result. Use this when you need to mathematically evaluate $\sqrt{x^2 + y^2}$ without intermediate overflow or underflow.

6 Rounding and Truncation Functions

6.1 ceil(x)

Description: Returns the value of x rounded up to its nearest integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.3;
    auto result = std::ceil(x);
    std::cout << "ceil(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::ceil`, and outputs the result. Use this when you need to mathematically evaluate the value of x rounded up to its nearest integer.

6.2 floor(x)

Description: Returns the value of x rounded down to its nearest integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.8;
    auto result = std::floor(x);
    std::cout << "floor(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::floor`, and outputs the result. Use this when you need to mathematically evaluate the value of x rounded down to its nearest integer.

6.3 `trunc(x)`

Description: Returns the integer part of x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.8;
    auto result = std::trunc(x);
    std::cout << "trunc(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::trunc`, and outputs the result. Use this when you need to mathematically evaluate the integer part of x.

6.4 `round(x)`

Description: Returns x rounded to the nearest integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::round(x);
    std::cout << "round(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::round`, and outputs the result. Use this when you need to mathematically evaluate x rounded to the nearest integer.

6.5 `lround(x)`

Description: Rounds x to the nearest integer and returns the result as a `long` integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::lround(x);
    std::cout << "lround(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::lround`, and outputs the result. Use this when you need to mathematically evaluate the rounds x to the nearest integer and result as a long integer.

6.6 `llround(x)`

Description: Rounds x to the nearest integer and returns the result as a `long long` integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::llround(x);
    std::cout << "llround(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::llround`, and outputs the result. Use this when you need to mathematically evaluate the rounds x to the nearest integer and result as a long long integer.

6.7 `rint(x)`

Description: Returns x rounded to a nearby integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::rint(x);
    std::cout << "rint(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::rint`, and outputs the result. Use this when you need to mathematically evaluate x rounded to a nearby integer.

6.8 `lrint(x)`

Description: Rounds x to a nearby integer and returns the result as a `long` integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::lrint(x);
    std::cout << "lrint(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::lrint`, and outputs the result. Use this when you need to mathematically evaluate the rounds x to a nearby integer and result as a long integer.

6.9 `llrint(x)`

Description: Rounds x to a nearby integer and returns the result as a `long long` integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::llrint(x);
    std::cout << "llrint(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::llrint`, and outputs the result. Use this when you need to mathematically evaluate the rounds x to a nearby integer and result as a long long integer.

6.10 `nearbyint(x)`

Description: Returns x rounded to a nearby integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.5;
    auto result = std::nearbyint(x);
    std::cout << "nearbyint(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::nearbyint`, and outputs the result. Use this when you need to mathematically evaluate x rounded to a nearby integer.

7 Minimum, Maximum, and Difference

7.1 `fdim(x, y)`

Description: Returns the positive difference between x and y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.0, y = 3.0;
    auto result = std::fdim(x, y);
    std::cout << "fdim(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::fdim`, and outputs the result. Use this when you need to mathematically evaluate the positive difference between x and y.

7.2 fmax(x, y)

Description: Returns the highest value of a floating x and y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.0, y = 3.0;
    auto result = std::fmax(x, y);
    std::cout << "fmax(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::fmax`, and outputs the result. Use this when you need to mathematically evaluate the highest value of a floating x and y.

7.3 fmin(x, y)

Description: Returns the lowest value of a floating x and y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.0, y = 3.0;
    auto result = std::fmin(x, y);
    std::cout << "fmin(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::fmin`, and outputs the result. Use this when you need to mathematically evaluate the lowest value of a floating x and y.

7.4 fma(x, y, z)

Description: Returns $x \times y + z$ without losing precision

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0, y = 3.0, z = 4.0;
    auto result = std::fma(x, y, z);
    std::cout << "fma(x, y, z) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::fma`, and outputs the result. Use this when you need to mathematically evaluate $x \times y + z$ without losing precision.

8 Remainder Functions

8.1 `fmod(x, y)`

Description: Returns the floating point remainder of x/y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.5, y = 2.0;
    auto result = std::fmod(x, y);
    std::cout << "fmod(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::fmod`, and outputs the result. Use this when you need to mathematically evaluate the floating point remainder of x/y.

8.2 `remainder(x, y)`

Description: Return the remainder of x/y rounded to the nearest integer

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.5, y = 2.0;
    auto result = std::remainder(x, y);
    std::cout << "remainder(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::remainder`, and outputs the result. Use this when you need to mathematically evaluate the return the remainder of x/y rounded to the nearest integer.

8.3 `remquo(x, y, z)`

Description: Calculates x/y rounded to the nearest integer, writes the result to the memory at the pointer z and returns the remainder.

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 10.3, y = 4.5;
    int quo;
    auto result = std::remquo(x, y, &quo);
    std::cout << "remquo(x, y, z) -> " << result << " (remainder), quotient: " << quo << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::remquo`, and outputs the result. Use this when you need to mathematically evaluate the calculates x/y rounded to the nearest integer, writes the result to the memory at the pointer z and remainder.

9 Floating-Point Manipulation

9.1 `copysign(x, y)`

Description: Returns the first floating point x with the sign of the second floating point y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 10.0, y = -1.0;
    auto result = std::copysign(x, y);
    std::cout << "copysign(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::copysign`, and outputs the result. Use this when you need to mathematically evaluate the first floating point x with the sign of the second floating point y.

9.2 `frexp(x, y)`

Description: With x expressed as $m \times 2^n$, returns the value of m (a value between 0.5 and 1.0) and writes the value of n to the memory at the pointer y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 8.0;
    int exp;
    auto result = std::frexp(x, &exp);
    std::cout << "frexp(x, y) -> " << result << " (mantissa), exponent:
    " << exp << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::frexp`, and outputs the result. Use this when you need to mathematically evaluate the with x expressed as $m \times 2^n$, value of m (a value between 0.5 and 1.0) and writes the value of n to the memory at the pointer y.

9.3 `ldexp(x, y)`

Description: Returns $x \times 2^y$

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0;
    int y = 3;
    auto result = std::ldexp(x, y);
    std::cout << "ldexp(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::ldexp`, and outputs the result. Use this when you need to mathematically evaluate $x \times 2^y$.

9.4 `modf(x, y)`

Description: Returns the decimal part of x and writes the integer part to the memory at the pointer y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 3.14159;
    double intpart;
    auto result = std::modf(x, &intpart);
    std::cout << "modf(x, y) -> " << result << " (fraction), integer
   part: " << intpart << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::modf`, and outputs the result. Use this when you need to mathematically evaluate the decimal part of x and writes the integer part to the memory at the pointer y.

9.5 `scalbn(x, y)`

Description: Returns $x \times R^y$ (R is usually 2)

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0;
    int y = 3;
    auto result = std::scalbn(x, y);
    std::cout << "scalbn(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::scalbn`, and outputs the result. Use this when you need to mathematically evaluate $x \times R^y$ (R is usually 2).

9.6 scalbln(x, y)

Description: Returns $x \times R^y$ (R is usually 2)

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 2.0;
    long y = 3;
    auto result = std::scalbln(x, y);
    std::cout << "scalbln(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::scalbln`, and outputs the result. Use this when you need to mathematically evaluate $x \times R^y$ (R is usually 2).

9.7 nextafter(x, y)

Description: Returns the closest floating point number to x in the direction of y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0, y = 2.0;
    auto result = std::nextafter(x, y);
    std::cout << "nextafter(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::nextafter`, and outputs the result. Use this when you need to mathematically evaluate the closest floating point number to x in the direction of y.

9.8 nexttoward(x, y)

Description: Returns the closest floating point number to x in the direction of y

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    long double y = 2.0;
    auto result = std::nexttoward(x, y);
    std::cout << "nexttoward(x, y) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::nexttoward`, and outputs the result. Use this when you need to mathematically evaluate the closest floating point number to x in the direction of y.

10 Error and Gamma Functions

10.1 erf(x)

Description: Returns the value of the error function at x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::erf(x);
    std::cout << "erf(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::erf`, and outputs the result. Use this when you need to mathematically evaluate the value of the error function at x.

10.2 erfc(x)

Description: Returns the value of the complementary error function at x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 1.0;
    auto result = std::erfc(x);
    std::cout << "erfc(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::erfc`, and outputs the result. Use this when you need to mathematically evaluate the value of the complementary error function at x.

10.3 tgamma(x)

Description: Returns the value of the gamma function at x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.0;
    auto result = std::tgamma(x);
    std::cout << "tgamma(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::tgamma`, and outputs the result. Use this when you need to mathematically evaluate the value of the gamma function at x.

10.4 `lgamma(x)`

Description: Returns the logarithm of the absolute value of the gamma function at x

```cpp
#include <iostream>
#include <cmath>

int main() {
    double x = 5.0;
    auto result = std::lgamma(x);
    std::cout << "lgamma(x) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::lgamma`, and outputs the result. Use this when you need to mathematically evaluate the logarithm of the absolute value of the gamma function at x.

11 Other Functions

11.1 `nan(s)`

Description: Returns a NaN (Not a Number) value

```cpp
#include <iostream>
#include <cmath>

int main() {
    const char* s = "1";
    auto result = std::nan(s);
    std::cout << "nan(s) -> " << result << std::endl;
    return 0;
}
```

Explanation: This snippet initializes the required variables, calls `std::nan`, and outputs the result. Use this when you need to mathematically evaluate a NaN (Not a Number) value.